Digital Audio Signal Generation, Filtering, and PWM-Based Speaker Drive Using FPGA

Final Design Report

Submitted by:
Aamaina Mukarram Tahir Ali, Syeda Alishba Zaidi & Huzaifa Ahmed Khan

Table of Contents

1. **Introduction**
2. **System Overview**
3. **FIR Low-Pass Filter Design in MATLAB**
    - 3.1 Filter Specifications
    - 3.2 Coefficients
    - 3.3 Frequency Response
    - 3.4 Impulse Response
    - 3.5 Filtering Operation
4. **Verilog Implementation of FIR Filter**
    - 4.1 Verilog Code of FIR Filter Module
    - 4.2 Simulation Waveforms
5. **ROM-Based Tones**
    - 5.1 ROM Schematic
    - 5.2 Logic and Implementation
    - 5.3 Simulation Waveforms
6. **DDS-Generated Tones**
    - 6.1 Phase Accumulator
    - 6.2 Phase Register for LUT Addressing
    - 6.3 Sine Look-Up Table
    - 6.4 Sine Output Update
7. **Convert FIR Output to PWM**
8. **Speaker Drive Tests**
9. **Verilog and Hardware Integration**

1. Introduction

This project implements a digital audio processing chain on an FPGA using:

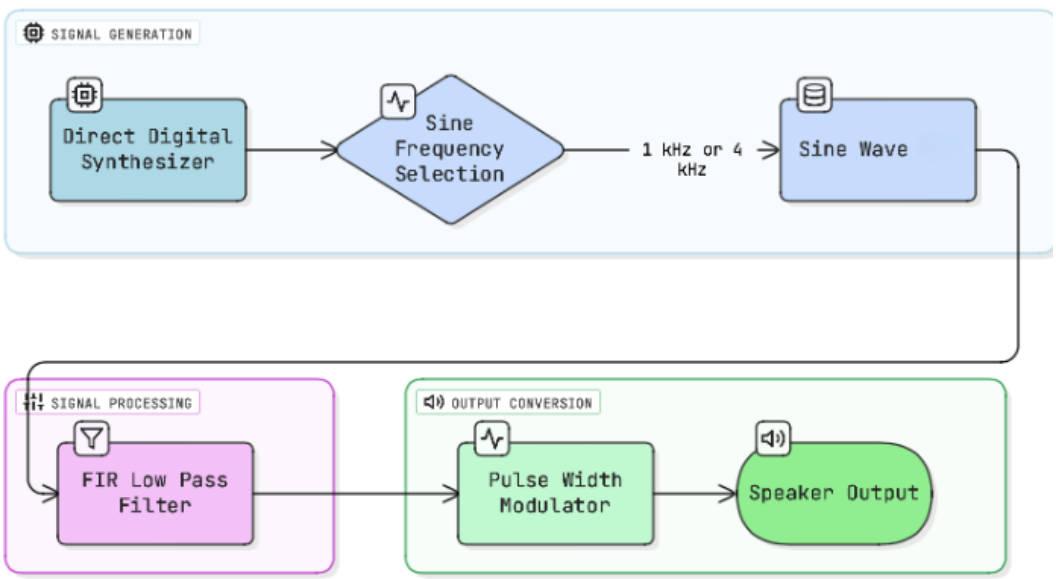
- MATLAB-designed FIR Low-Pass Filter
- Direct Digital Synthesis (DDS)
- Real-time convolution using Verilog
- PWM-driven speaker output

The objective is to design a system able to:

1. Generate digital sine wave signals
2. Filter them through a 1 kHz low-pass FIR filter
3. Output the filtered audio using PWM
4. Validate behavior using ROM-based tests and real-time DDS input
5. Demonstrate real audible difference between passband and stopband signals

This report documents the full design flow, simulations, MATLAB generation, Verilog modules, FPGA testing, and hardware results.

2. System-Level Block Diagram



3. FIR Low-Pass Filter Design in MATLAB

FIR (Finite Impulse Response) filtering is a method to process signals by convolving the input with a set of coefficients. Each coefficient determines the weight of past input samples in producing the output. Key specifications for the design include the Sampling Frequency, which defines the rate of input signal acquisition; the Cutoff Frequency, determining the passband edge; the Filter Order, influencing the sharpness of the frequency transition; the Window type, utilized to shape the coefficient set and mitigate undesired spectral effects; and the Coefficient Quantization level, which prepares the coefficients for digital hardware implementation.

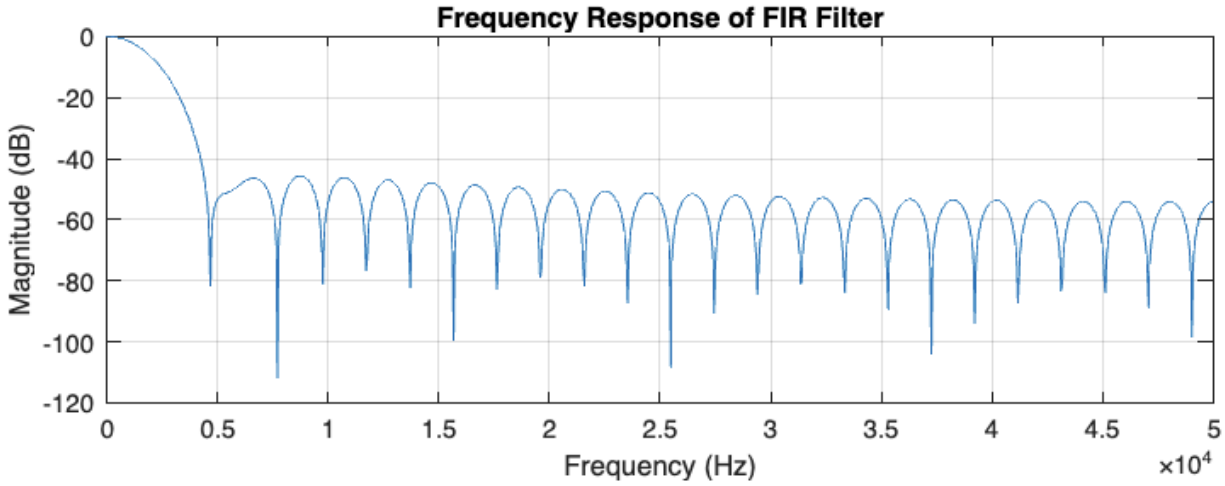
3.1 Filter Specifications

Parameter	Value
Sampling Frequency (Fs)	100 kHz
Passband	0–1 kHz
Transition band	1 kHz → 3 kHz
Stopband	> 3 kHz
Stopband attenuation	≈ −40 dB
Filter order	50
No. of taps	51
Window	Hamming
Coefficient format	8-bit signed

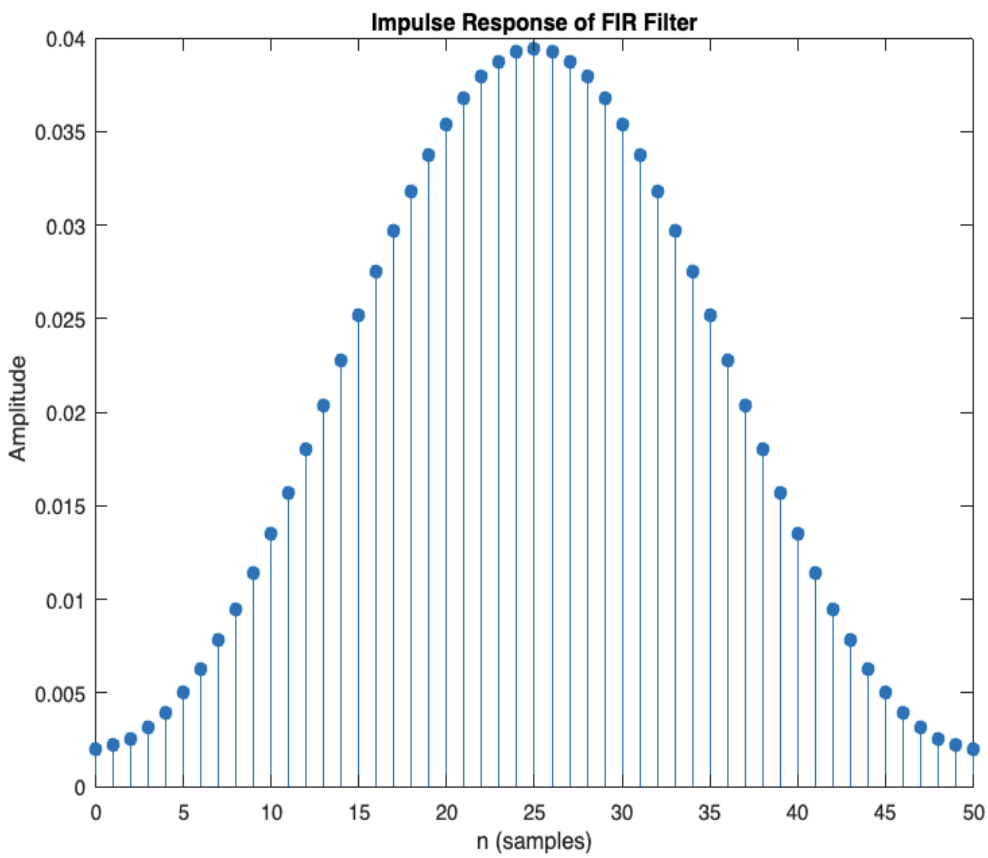
3.2 Coefficients

```
FIR Coefficients:
    0.0020    0.0022    0.0026    0.0031    0.0040    0.0050    0.0063    0.0078    0.0095    0.0114    0.0135    0.0157

8-bit signed coefficients:
     6     7     8    10    13    16    20    25    31    37    43    50    58    66    73    81    89    96   102   109
```

3.3 Frequency Response

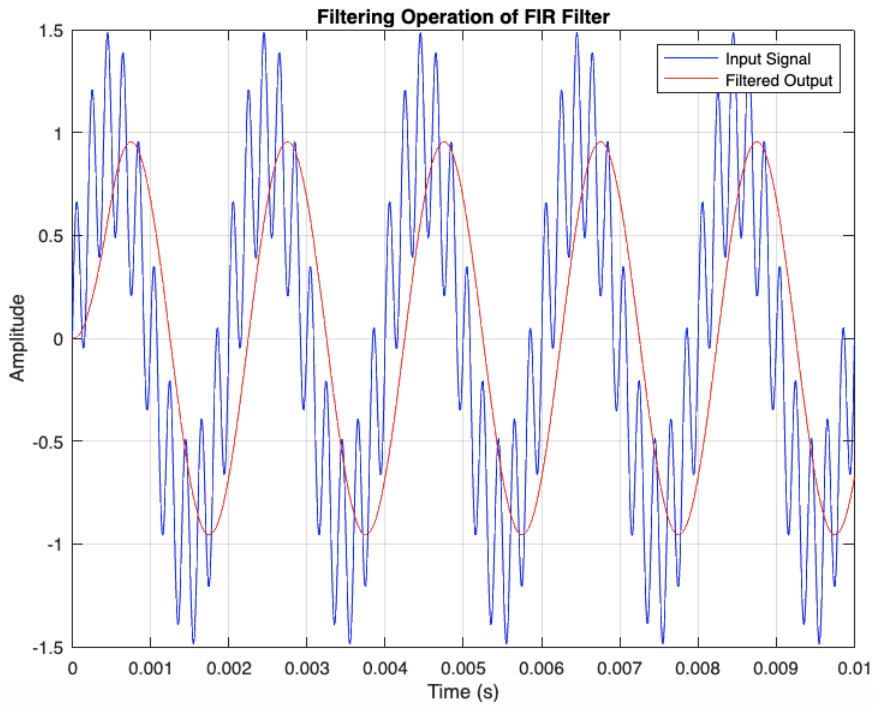


3.4 Impulse Response



3.5 Filtering operation

The FIR filter performs convolution with input samples, producing a filtered output suitable for PWM conversion



4. Verilog Implementation of FIR Filter

4.1 Verilog Code of FIR Filter Module

This Verilog module implements a 51-tap low-pass FIR filter whose coefficients were generated in MATLAB using a Hamming-windowed FIR design. The filter was designed for a 100 kHz sampling rate, 1 kHz cutoff, and a 1–3 kHz transition band, and all coefficients were quantized to 8-bit signed integers (range −128 to +127). These quantized values appear in the module as b0 through b50, arranged symmetrically to maintain the linear-phase property of the FIR filter.

```
parameter order = 51;
parameter word_size_in  = 8;
parameter word_size_out = 16;
```

Inputs and Outputs

- Data_in (8-bit signed): The new input sample received every clock cycle.
- clk: 100 kHz processing clock.
- reset: Clears internal registers.
- Data_out (16-bit signed): The filtered output sample, generated once per clock.

```
module fir_filter (
    output reg signed [15:0] Data_out,
    input  signed [7:0] Data_in,
    input clk,
    input reset
);
```

Internal Logic

The filter implements the standard FIR convolution equation:

$$y[n] = \Sigma\ (b[k] * x[n - k]) \quad \text{for } k = 0 \text{ to } 50$$

To compute this in hardware, the module uses three main blocks:

1. **Delay Line (Samples[]):**
   A shift register of 51 entries that stores the most recent input samples.
   Every clock cycle, all samples shift right and the new input sample is stored at Samples[0].

```
always @(posedge clk) begin
    if (reset) begin
        for (k = 0; k < order; k = k + 1)
            Samples[k] <= 0;
    end else begin
        for (k = order-1; k > 0; k = k - 1)
            Samples[k] <= Samples[k-1];
        Samples[0] <= Data_in;
    end
end
```

2. **Parallel Multipliers (prod[]):**
   Each stored sample is multiplied by its corresponding coefficient bk.
   This results in 51 signed 16-bit products.

3. **Accumulator (adder chain):**
   All 51 products are sign-extended to 24 bits and added together in one long summation chain.
   The result is a high-precision 24-bit value.

Finally, the module outputs:

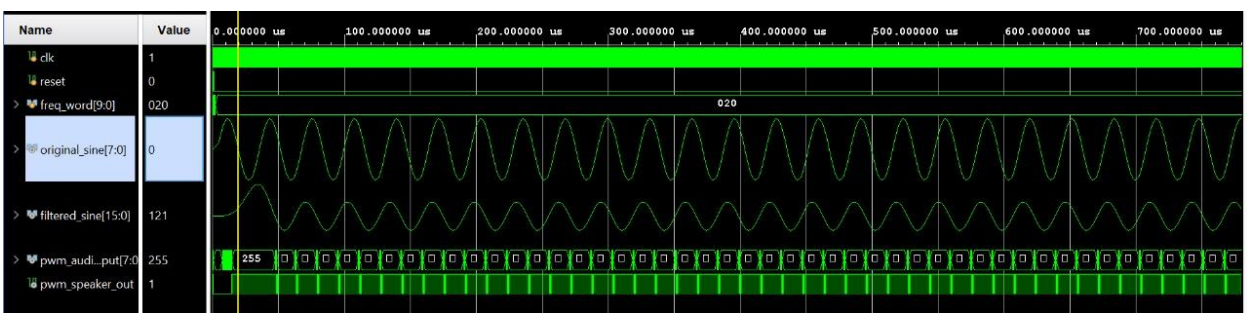
Data_out=accumulator[23:8]

```
always @(posedge clk) begin
    if (reset) begin
        Data_out <= 0;
    end else begin
        Data_out <= accumulator[23:8];
    end
end
```

This selects the upper bits to avoid overflow and to scale the result back down after filtering.

4.2 Simulation Waveforms

1. **Low Frequency Input (500 Hz)**



Observation: The filter output closely follows the DDS signal. PWM duty cycle changes smoothly. Audio is clearly audible.

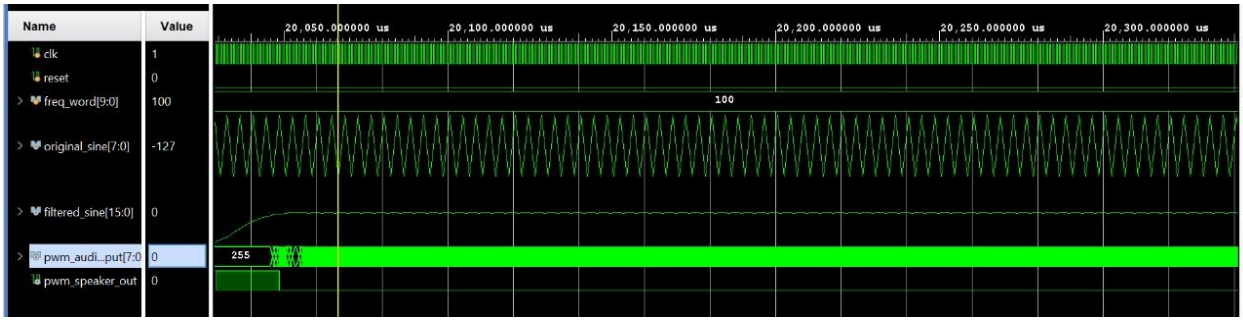
Numerical sample:

- Input: 0
- Filtered output: 121
- PWM: 255 (maximum)

Analysis: The 500 Hz signal lies within the passband. Filter output remains high even near zero crossings due to the

impulse response and group delay. Maximum PWM duty cycle confirms minimal attenuation.
Conclusion: Low-frequency signals pass effectively through the FIR filter, producing clear audio.

2. **Medium Frequency Input (4kHz)**



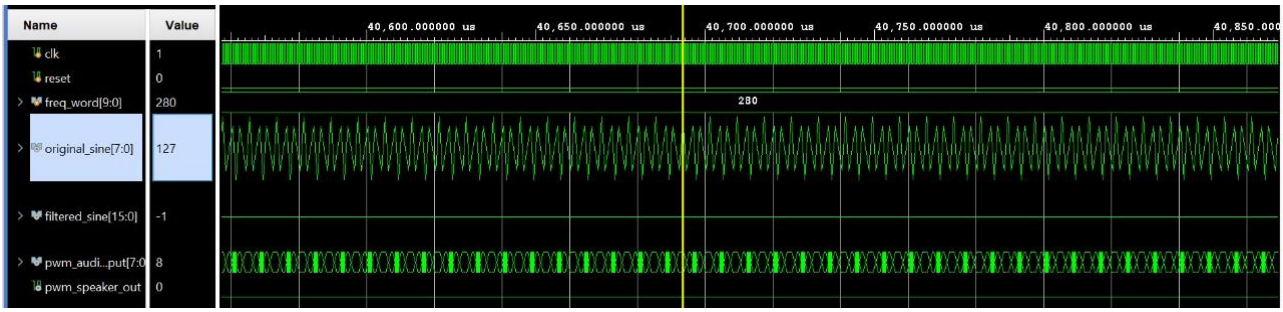
Observation: Filter output amplitude reduces significantly. PWM becomes nearly constant. Audio is faint.

Numerical sample:

- Input: -127
- Filtered output: 0
- PWM: 0

Analysis: The 4 kHz signal lies near the stopband. Output is fully attenuated, and PWM output shows zero duty
cycle.
Conclusion: The FIR filter attenuates frequencies above the cutoff effectively

3. **High Frequency Input (~10 kHz)**



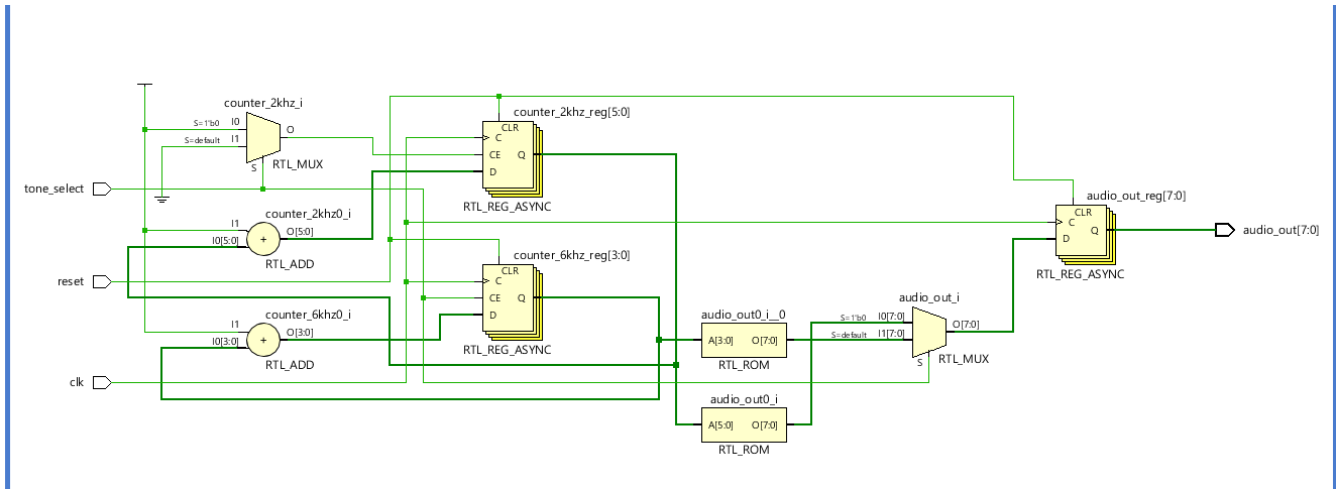
Observation: Filter output is near zero. PWM remains mostly steady. No audible output.
Numerical sample:

- Input: 127
- Filtered output: -1

Analysis: 10 kHz is far beyond the cutoff frequency. Approximate attenuation:
`20 × log10(1 / 127) ≈ −42 dB`
Conclusion: `The filter successfully blocks high-frequency content, meeting the −40 dB stopband specification.`

5. ROM-Based Tones

5.1 ROM Schematic:



5.2 Logic and Implementation

We have implemented a ROM module to store and generate fixed-frequency sine waves, serving as a signal source for the FIR filter. Two separate look-up tables were created: a 64-sample ROM for a 2 kHz tone and a 16-sample ROM for a 6 kHz tone, both containing 8-bit signed values pre-computed from a sine function scaled to the range -128 to +127. Each ROM stores exactly one complete cycle of the respective waveform, enabling periodic playback by circularly reading through the stored samples. The module uses independent counters (counter_2khz[5:0] and counter_6khz[3:0]) that increment at the 100 kHz system clock rate; the 1 kHz tone cycles through all 64 samples per period (achieving 2 kHz at 100 kHz sampling), while the 6 kHz tone cycles through 16 samples per period. A tone_select input determines which ROM is active, with the selected 8-bit sample output (audio_out) fed directly into the FIR filter's Data_in port for real-time filtering. This ROM-based approach provides a stable, deterministic signal source ideal for initial filter verification and frequency-response testing without requiring a DDS phase accumulator.

```
`timescale 1ns / 1ps

module tone_rom (
    input wire clk,                 // 100kHz clock
    input wire reset,               // Active high reset
    input wire tone_select,         // 0 = 2kHz, 1 = 6kHz
    output reg signed [7:0] audio_out  // 8-bit signed audio output
);

    // Internal counters
    reg [5:0] counter_2khz;  // 0 to 63 (64 samples)
    reg [3:0] counter_6khz;  // 0 to 15 (16 samples)

    // ROM for 2kHz tone (64 samples of one sine wave cycle)
    reg signed [7:0] rom_2khz [0:63];

    // ROM for 6kHz tone (16 samples of one sine wave cycle)
    reg signed [7:0] rom_6khz [0:15];

```

Look-up table for 2kHz Tone ROM

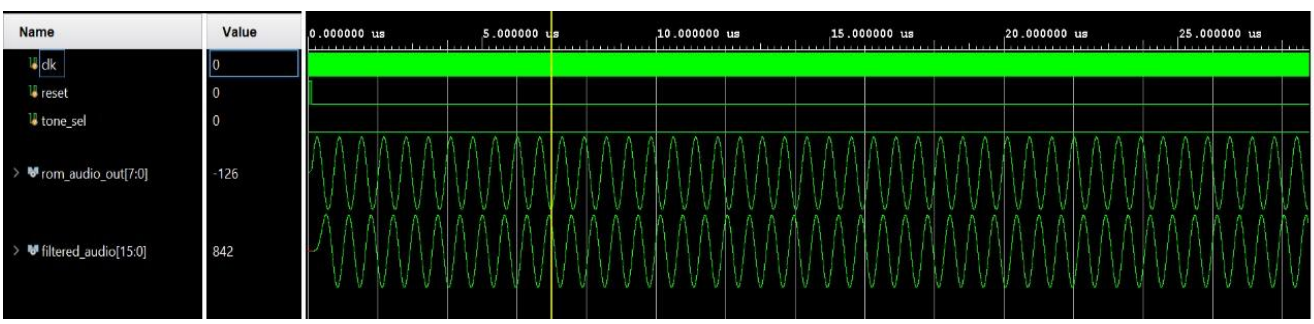
```
    initial begin
        // 2kHz Tone ROM (64 samples) - Already calculated values
        rom_2khz[0] = 8'd0;       rom_2khz[32] = 8'd0;
        rom_2khz[1] = 8'd12;      rom_2khz[33] = -8'd12;
        rom_2khz[2] = 8'd25;      rom_2khz[34] = -8'd25;
        rom_2khz[3] = 8'd37;      rom_2khz[35] = -8'd37;
        rom_2khz[4] = 8'd49;      rom_2khz[36] = -8'd49;
        rom_2khz[5] = 8'd60;      rom_2khz[37] = -8'd60;
        rom_2khz[6] = 8'd71;      rom_2khz[38] = -8'd71;
        rom_2khz[7] = 8'd81;      rom_2khz[39] = -8'd81;
        rom_2khz[8] = 8'd90;      rom_2khz[40] = -8'd90;
        rom_2khz[9] = 8'd98;      rom_2khz[41] = -8'd98;
        rom_2khz[10] = 8'd106;    rom_2khz[42] = -8'd106;
        rom_2khz[11] = 8'd112;    rom_2khz[43] = -8'd112;
        rom_2khz[12] = 8'd117;    rom_2khz[44] = -8'd117;
        rom_2khz[13] = 8'd121;    rom_2khz[45] = -8'd121;
        rom_2khz[14] = 8'd124;    rom_2khz[46] = -8'd124;
        rom_2khz[15] = 8'd126;    rom_2khz[47] = -8'd126;
        rom_2khz[16] = 8'd127;    rom_2khz[48] = -8'd127;
        rom_2khz[17] = 8'd126;    rom_2khz[49] = -8'd126;
        rom_2khz[18] = 8'd124;    rom_2khz[50] = -8'd124;
        rom_2khz[19] = 8'd121;    rom_2khz[51] = -8'd121;
        rom_2khz[20] = 8'd117;    rom_2khz[52] = -8'd117;
        rom_2khz[21] = 8'd112;    rom_2khz[53] = -8'd112;
        rom_2khz[22] = 8'd106;    rom_2khz[54] = -8'd106;
        rom_2khz[23] = 8'd98;     rom_2khz[55] = -8'd98;
        rom_2khz[24] = 8'd90;     rom_2khz[56] = -8'd90;
        rom_2khz[25] = 8'd81;     rom_2khz[57] = -8'd81;
        rom_2khz[26] = 8'd71;     rom_2khz[58] = -8'd71;
        rom_2khz[27] = 8'd60;     rom_2khz[59] = -8'd60;
        rom_2khz[28] = 8'd49;     rom_2khz[60] = -8'd49;
        rom_2khz[29] = 8'd37;     rom_2khz[61] = -8'd37;
        rom_2khz[30] = 8'd25;     rom_2khz[62] = -8'd25;
        rom_2khz[31] = 8'd12;     rom_2khz[63] = -8'd12;
```

Look-up table for 6kHz Tone ROM

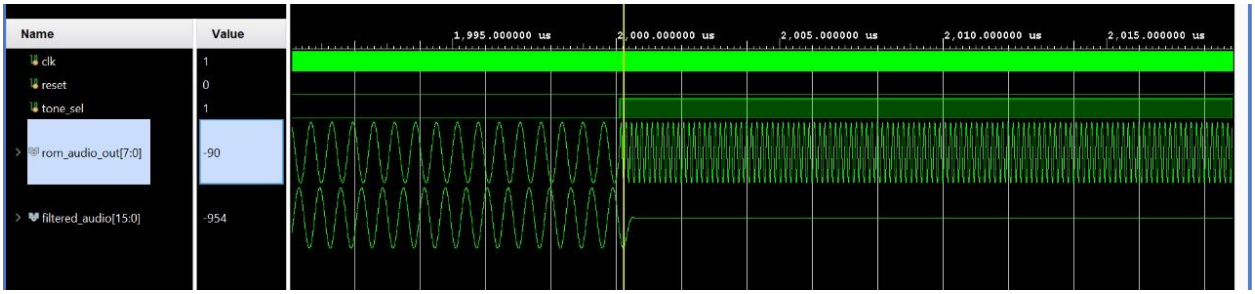
```
        // 6kHz Tone ROM (16 samples)
        rom_6khz[0] = 8'd0;       rom_6khz[8] = 8'd0;
        rom_6khz[1] = 8'd49;      rom_6khz[9] = -8'd49;
        rom_6khz[2] = 8'd90;      rom_6khz[10] = -8'd90;
        rom_6khz[3] = 8'd117;     rom_6khz[11] = -8'd117;
        rom_6khz[4] = 8'd127;     rom_6khz[12] = -8'd127;
        rom_6khz[5] = 8'd117;     rom_6khz[13] = -8'd117;
        rom_6khz[6] = 8'd90;      rom_6khz[14] = -8'd90;
        rom_6khz[7] = 8'd49;      rom_6khz[15] = -8'd49;
    end

    // ================================================
    // MAIN PROCESS: Read from ROM
    // ================================================
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            counter_2khz <= 0;
            counter_6khz <= 0;
            audio_out <= 0;
        end else begin
            if (tone_select == 1'b0) begin
                // 2kHz tone - read from 64-entry ROM
                counter_2khz <= counter_2khz + 1;
                audio_out <= rom_2khz[counter_2khz];
            end else begin
                // 6kHz tone - read from 16-entry ROM
                counter_6khz <= counter_6khz + 1;
                audio_out <= rom_6khz[counter_6khz];
            end
        end
    end
```

5.3 Simulation Waveforms

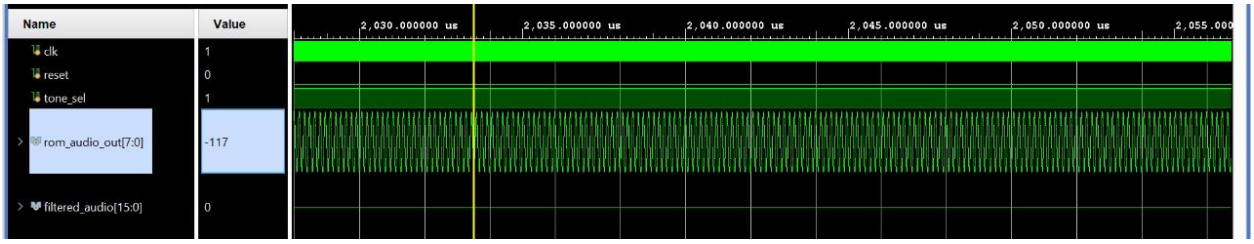
2 kHz when the tone selector is 0



The transition from 2 kHz to 6 kHz



6 kHz



6. DDS-Generated Tones

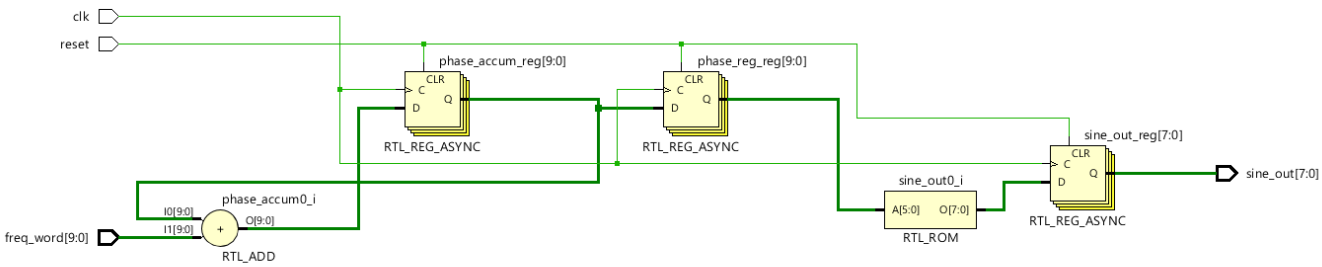
The dds module implements a Direct Digital Synthesis (DDS) sine-wave generator using a phase accumulator and a 64-entry sine Look-Up Table (LUT). It produces an 8-bit signed sine wave at a programmable output frequency, based on a 10-bit frequency control word.

Once the DDS generates each 8-bit sample, it becomes the direct input to the FIR filter. At every 100 kHz clock cycle, the newly generated sine sample from the DDS is fed into the Data_in port of the FIR filter module. The FIR delay line shifts this new sample into its tap structure while simultaneously performing 51 parallel multiplications with the pre-computed coefficients.

The DDS and FIR modules operate based on a 100 kHz processing clock, which is derived from the 100 MHz system clock using a clock divider. The clock divider counts 500 cycles of the 100 MHz system clock and toggles its output, producing a 100 kHz clock (clk_100kHz) for synchronous operation. This 100 kHz clock serves as the sampling rate for both the DDS and FIR filter, ensuring that each newly generated sine sample is processed precisely at this interval. Meanwhile, the PWM module continues to run at the full 100 MHz system clock, allowing it to modulate the filtered output with high resolution. By separating the slower processing clock for

sample generation from the high-speed PWM clock, the design achieves accurate waveform synthesis and filtering while maintaining precise output timing for audio applications.

Schematic:



Inputs and Outputs

Inputs

- clk (1 MHz):
  Master clock used for phase accumulation and sample updates.
  The output sine frequency depends directly on this and on freq_word.
- reset (active high):
  Clears the phase accumulator and output.
- freq_word (10-bit):
  Digital tuning word that controls output frequency.
  The DDS output frequency is:

fout=freq_word1024×fclk

With f_clk = 1 MHz, typical values are:

- freq_word = 102 → 1000 Hz
- freq_word = 409 → ~4000 Hz

Output

- sine_out (8-bit signed):
  The generated sine sample (range −128 to +127), updated every clock cycle.

Internal Logic Breakdown

6.1. Phase Accumulator

reg [9:0] phase_accum;

- This register increments by freq_word every clock.
- It represents the phase angle of the sine wave.
- When it overflows, the sine wave completes one cycle.

This implements the core DDS equation:

phase(n+1)=phase(n)+freq_word

6.2. Phase Register for LUT Addressing

phase_reg <= phase_accum;

- The upper 6 bits of phase_reg select the LUT entry.
- Using phase_reg[9:4] creates a 64-point sine wave.

```
always @(posedge clk or posedge reset) begin
    if (reset) begin
        phase_accum <= 10'b0;
        phase_reg   <= 10'b0;
        sine_out    <= 8'b0;
    end else begin
        // Phase accumulation
        phase_accum <= phase_accum + freq_word;
        phase_reg <= phase_accum;

        // Sine lookup using top 6 bits for 64-entry LUT
        sine_out <= sine_lut[phase_reg[9:4]];
    end
end
```

6.3. Sine Look-Up Table (LUT)

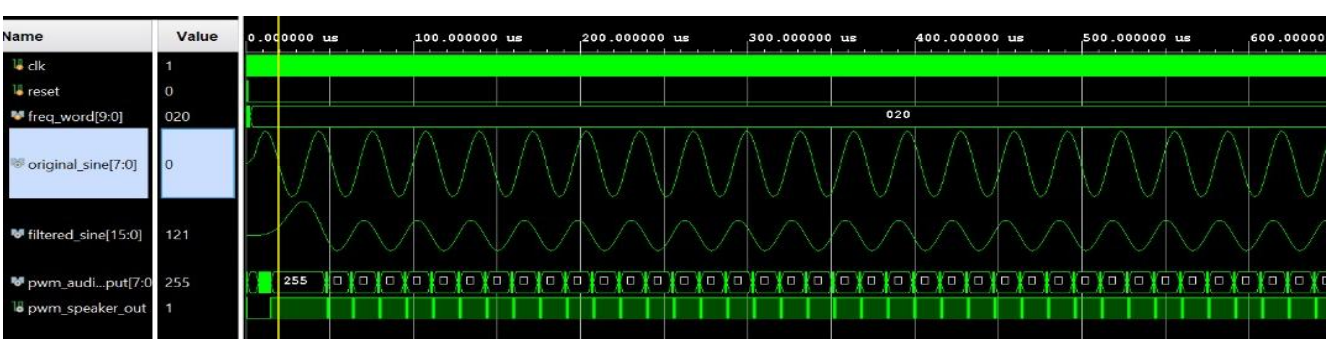
- Contains 64 samples of a full sine wave.
- Values range from −128 to +127, matching your FIR input width

```
// SINE LUT INITIALIZATION - Scaled to 8-bit signed range (-128 to 127)
initial begin
    // Quarter sine wave scaled to 8-bit signed
    sine_lut[0] = 8'd0;       sine_lut[1] = 8'd12;      sine_lut[2] = 8'd25;
    sine_lut[3] = 8'd37;      sine_lut[4] = 8'd49;      sine_lut[5] = 8'd60;
    sine_lut[6] = 8'd71;      sine_lut[7] = 8'd81;      sine_lut[8] = 8'd90;
    sine_lut[9] = 8'd98;      sine_lut[10] = 8'd106;    sine_lut[11] = 8'd112;
    sine_lut[12] = 8'd117;    sine_lut[13] = 8'd121;    sine_lut[14] = 8'd124;
    sine_lut[15] = 8'd126;    sine_lut[16] = 8'd127;    sine_lut[17] = 8'd126;
    sine_lut[18] = 8'd124;    sine_lut[19] = 8'd121;    sine_lut[20] = 8'd117;
    sine_lut[21] = 8'd112;    sine_lut[22] = 8'd106;    sine_lut[23] = 8'd98;
    sine_lut[24] = 8'd90;     sine_lut[25] = 8'd81;     sine_lut[26] = 8'd71;
    sine_lut[27] = 8'd60;     sine_lut[28] = 8'd49;     sine_lut[29] = 8'd37;
    sine_lut[30] = 8'd25;     sine_lut[31] = 8'd12;     sine_lut[32] = 8'd0;
    sine_lut[33] = -8'd12;    sine_lut[34] = -8'd25;    sine_lut[35] = -8'd37;
    sine_lut[36] = -8'd49;    sine_lut[37] = -8'd60;    sine_lut[38] = -8'd71;
    sine_lut[39] = -8'd81;    sine_lut[40] = -8'd90;    sine_lut[41] = -8'd98;
    sine_lut[42] = -8'd106;   sine_lut[43] = -8'd112;   sine_lut[44] = -8'd117;
    sine_lut[45] = -8'd121;   sine_lut[46] = -8'd124;   sine_lut[47] = -8'd126;
    sine_lut[48] = -8'd127;   sine_lut[49] = -8'd126;   sine_lut[50] = -8'd124;
    sine_lut[51] = -8'd121;   sine_lut[52] = -8'd117;   sine_lut[53] = -8'd112;
    sine_lut[54] = -8'd106;   sine_lut[55] = -8'd98;    sine_lut[56] = -8'd90;
    sine_lut[57] = -8'd81;    sine_lut[58] = -8'd71;    sine_lut[59] = -8'd60;
    sine_lut[60] = -8'd49;    sine_lut[61] = -8'd37;    sine_lut[62] = -8'd25;
    sine_lut[63] = -8'd12;
end
```

This removes the need for real-time trigonometric computation.

6.4. Sine Output Update

- Every clock, DDS outputs one sample from the LUT.
- Frequency depends on how fast the phase accumulator rolls over.



The DDS outputs a real-time 8-bit sine sample (0) using a 64-point LUT and frequency word 020h, while the FIR filter concurrently produces a 16-bit filtered result (121) due to its 51-tap convolution pipeline..

7. Convert FIR Output to PWM

The PWM generator converts the multi-bit filtered audio signal into a 1-bit pulse-width modulated output, which is suitable for directly driving an 8-ohm speaker on the Basys-3 board. The module compares a continuously increasing counter with the input audio sample to produce a variable-duty-cycle output representing the waveform amplitude. It allows direct speaker drive with no DAC and converts filtered digital samples into audible sound.

Inputs

- **clk:** System clock used to run the PWM counter.
- **reset:** Clears counter and PWM output.
- **audio_in:** The filtered multi-bit FIR output (typically 10–12 bits depending on design).

Output

- **pwm_out:**
  A single-bit PWM signal, where the duty cycle is proportional to the audio amplitude.
  The speaker averages this signal due to its natural mechanical low-pass behavior.

Internal Logic

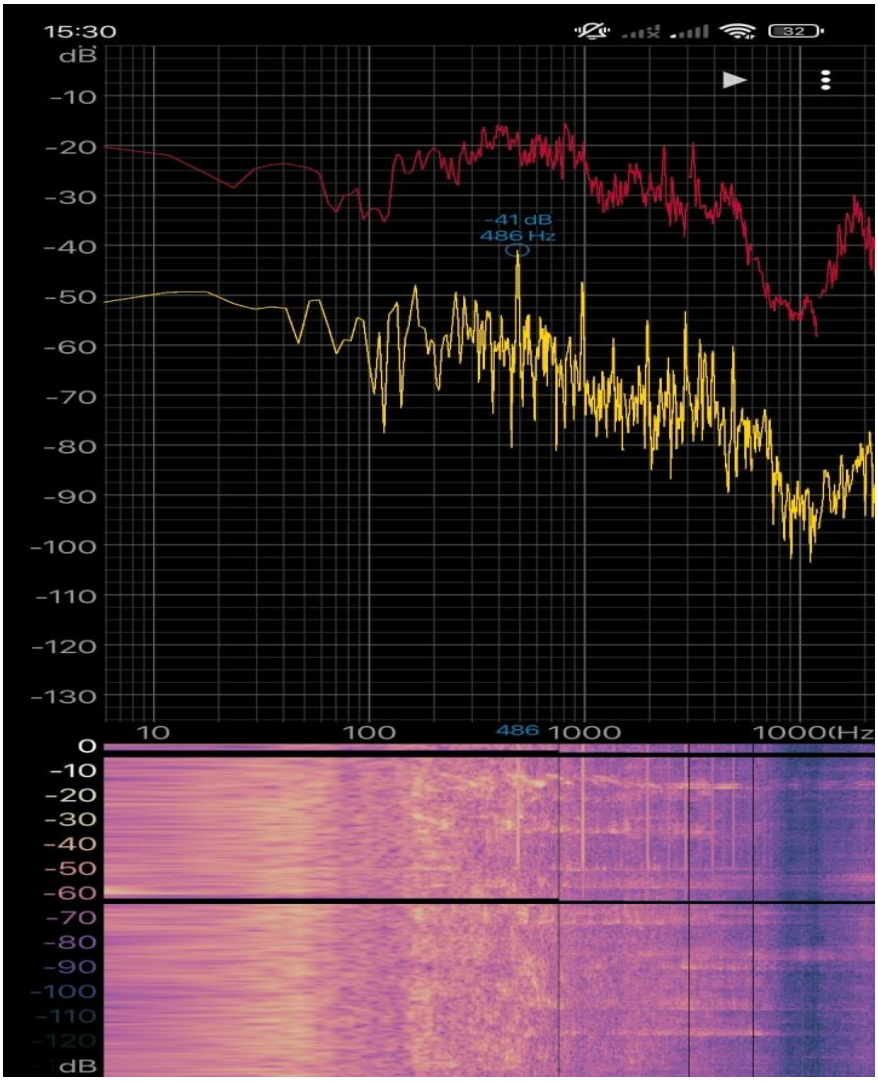
- A free-running counter ramps from 0 to its maximum value.
- On each clock:
  - If audio_in > counter → pwm_out = 1
  - Else → pwm_out = 0
- Larger input value → higher duty cycle → louder instantaneous sound pressure.

```
module pwm_generator_high_res (
    input wire clk,               // 100MHz clock
    input wire reset,
    input wire [7:0] audio_input, // 0-255 scaled audio
    output reg pwm_out
);

    reg [7:0] pwm_counter;

    always @(posedge clk) begin
        if (reset) begin
            pwm_counter <= 8'b0;
            pwm_out <= 1'b0;
        end else begin
            pwm_counter <= pwm_counter + 1;
            // Proper PWM: output HIGH when counter < audio_input value
            pwm_out <= (pwm_counter < audio_input);
        end
    end
```

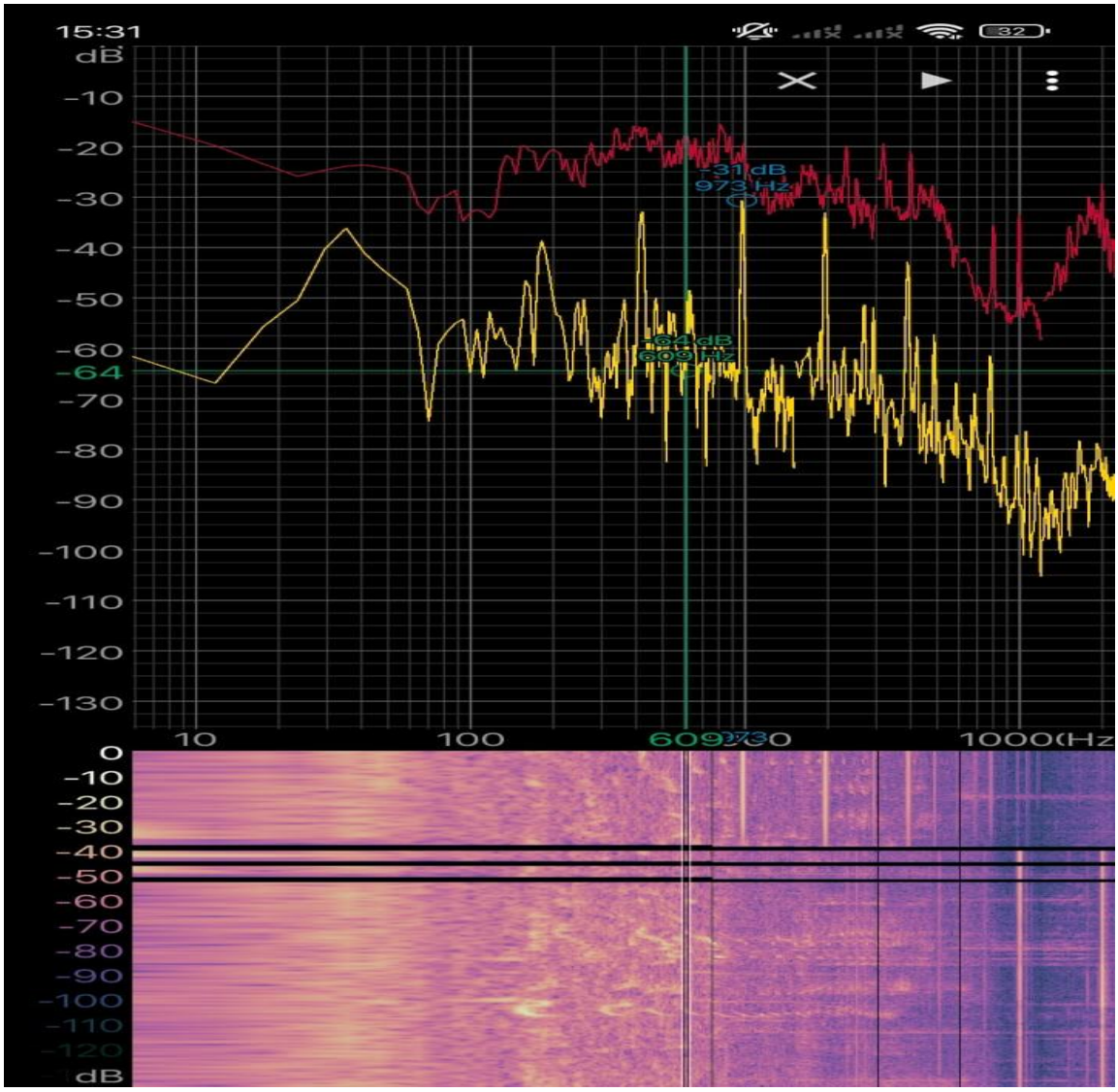
8. Speaker Drive Tests

Modes:

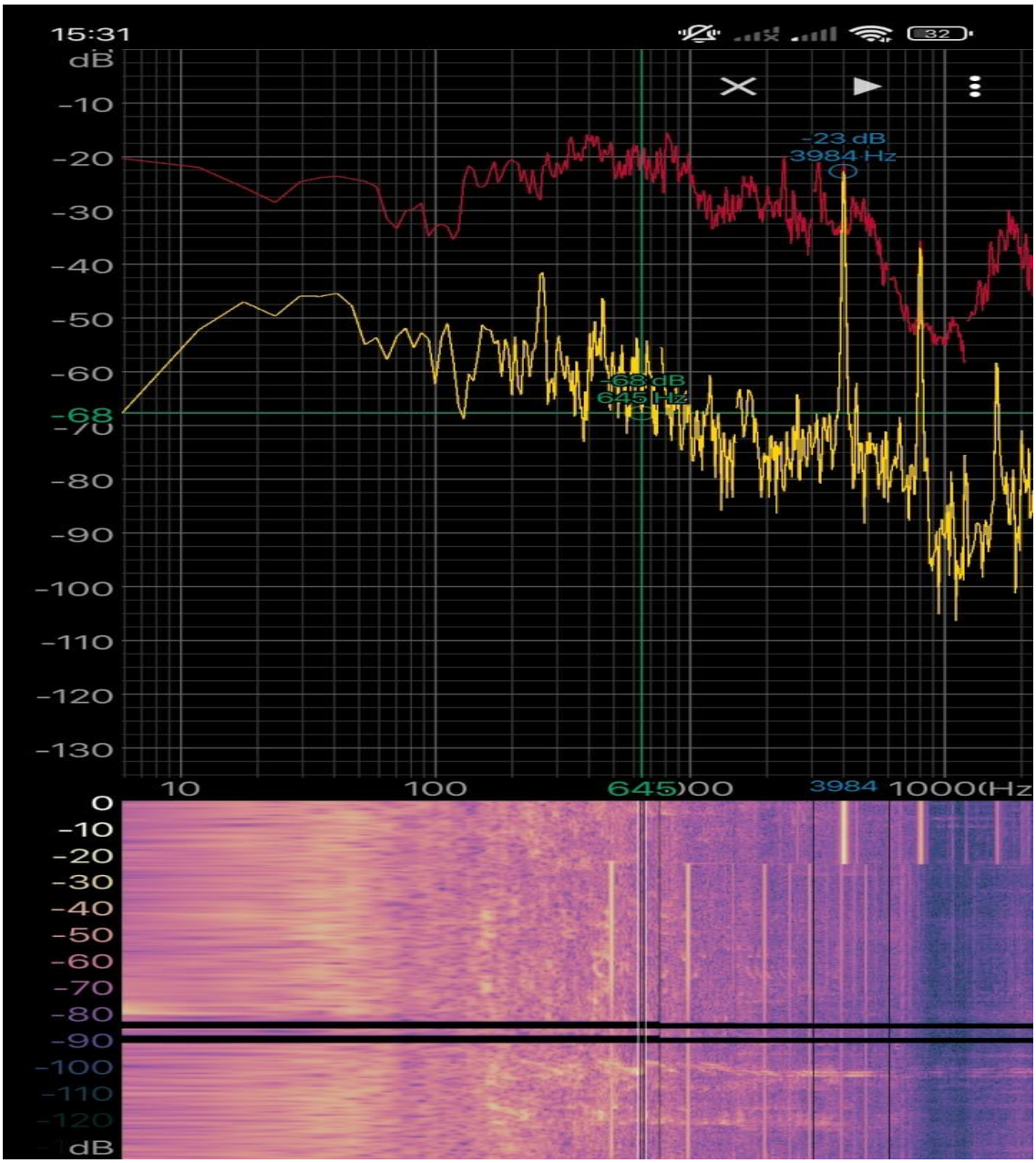
1. Unfiltered 500 Hz → audible



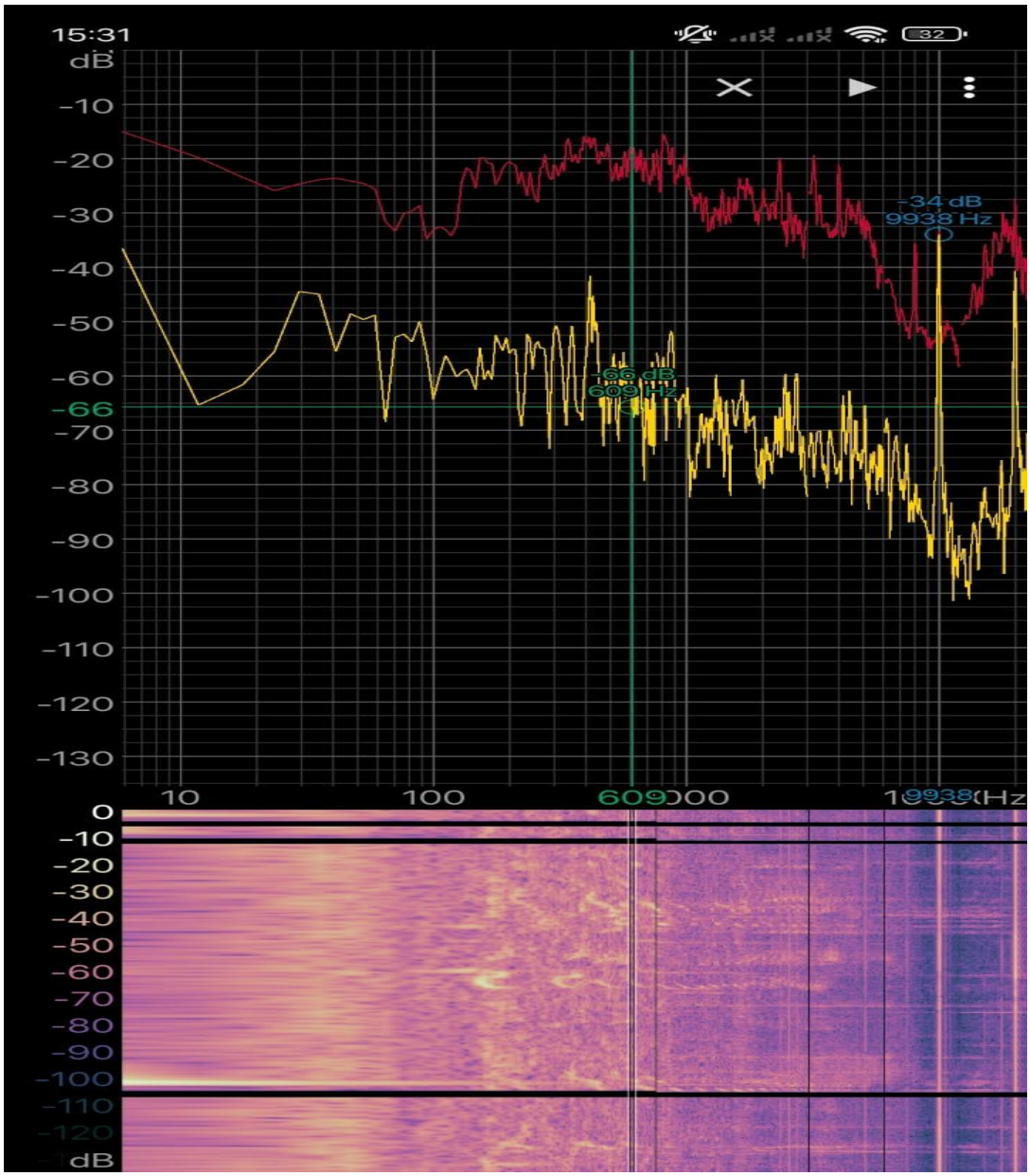
2. **Unfiltered 1kHz → Audible**



3. **Unfiltered 4 kHz → Audible**



4. Unfiltered 10 kHz → Audible



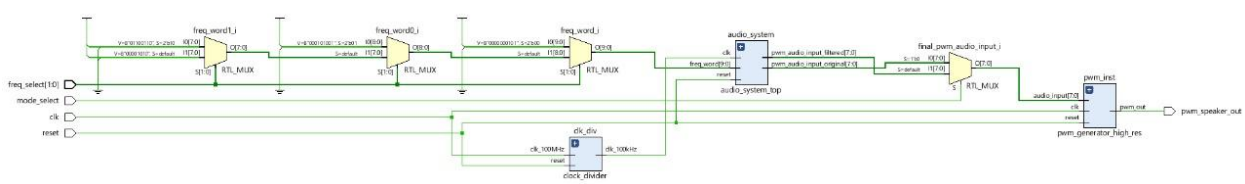
Results:
500 Hz and 1kHz were passing through the fir filter and attenuated noise could be heard on speaker, No noise on 4 kHz and 10 kHz

Amplification to hear on speaker:

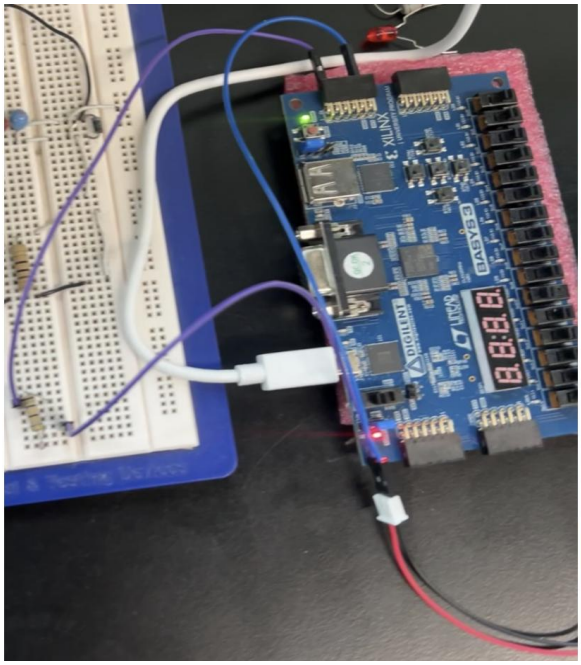
```
    // ================================================
    // AMPLIFICATION & PWM PREPARATION
    // ================================================

    // Original wave -> PWM input
    wire [7:0] original_unsigned = selected_wave + 8'd128;  // Convert to 0-255
    wire [15:0] original_amplified = original_unsigned * 8'd4;

    // Filtered wave -> PWM input (with careful scaling)
    wire signed [15:0] abs_filtered = (fir_filter_out[15]) ? (~fir_filter_out + 1) : fir_filter_out;
    wire [7:0] filtered_scaled;

    // Frequency-dependent scaling (same as before)
    assign filtered_scaled =
        (freq_word == 10'd5)   ? (abs_filtered[10:3]) :  // 500Hz
        (freq_word == 10'd10)  ? (abs_filtered[11:4]) :  // 1kHz
        (freq_word == 10'd41)  ? (abs_filtered[12:5]) :  // 4kHz
        (abs_filtered[13:6]);                            // 10kHz

    // Final PWM inputs (clipped to 0-255)
    assign pwm_audio_input_original = (original_amplified > 255) ? 8'd255 : original_amplified[7:0];
    assign pwm_audio_input_filtered = filtered_scaled;

    // Debug outputs
    assign original_wave = selected_wave;
    assign filtered_wave = fir_filter_out;

endmodule
```

Schematic of the system on Xilinx Vivado:

The system was deployed on FPGA hardware and connected to a speaker through a PWM output pin.



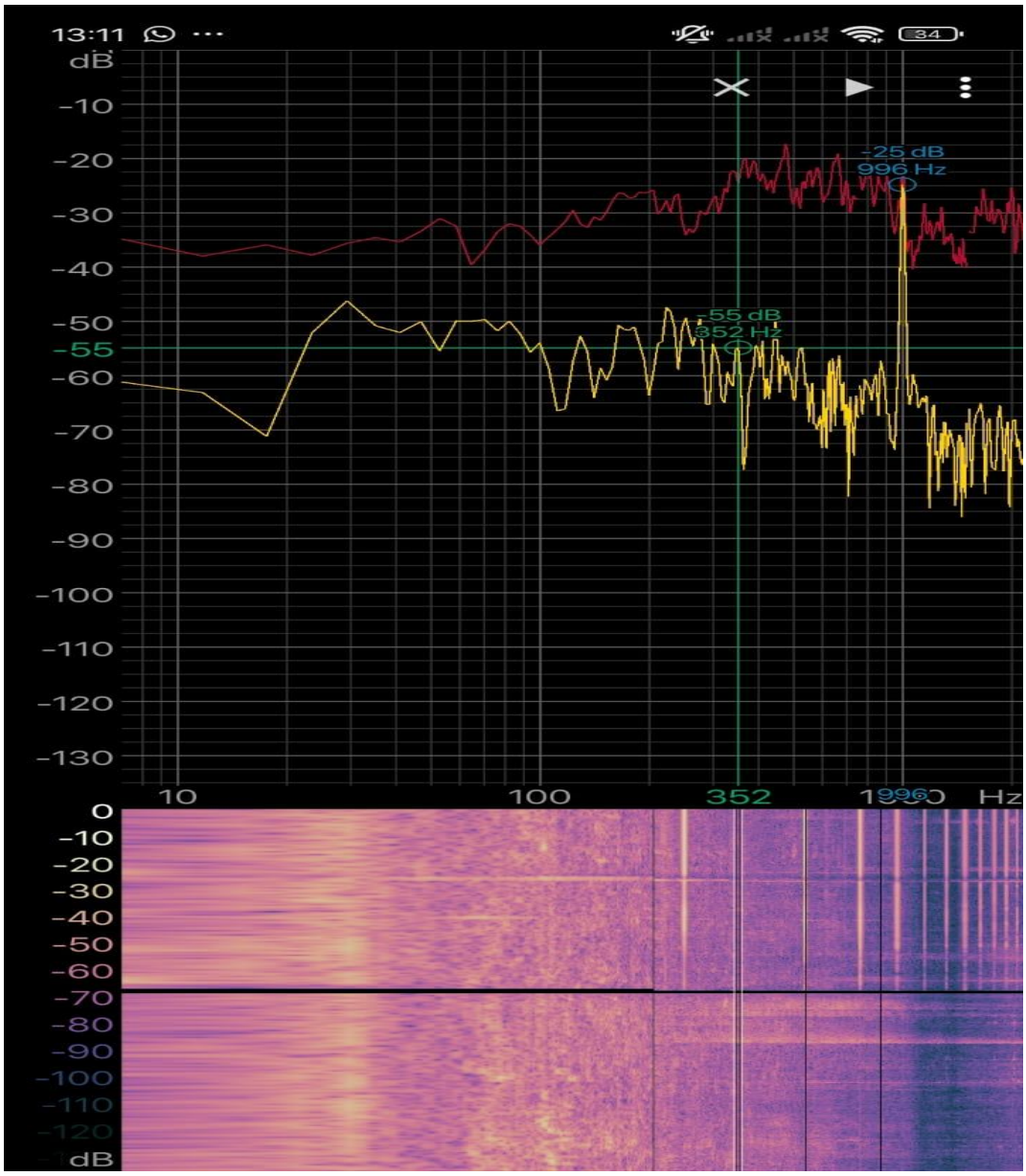
Verilog and Hardware Integration



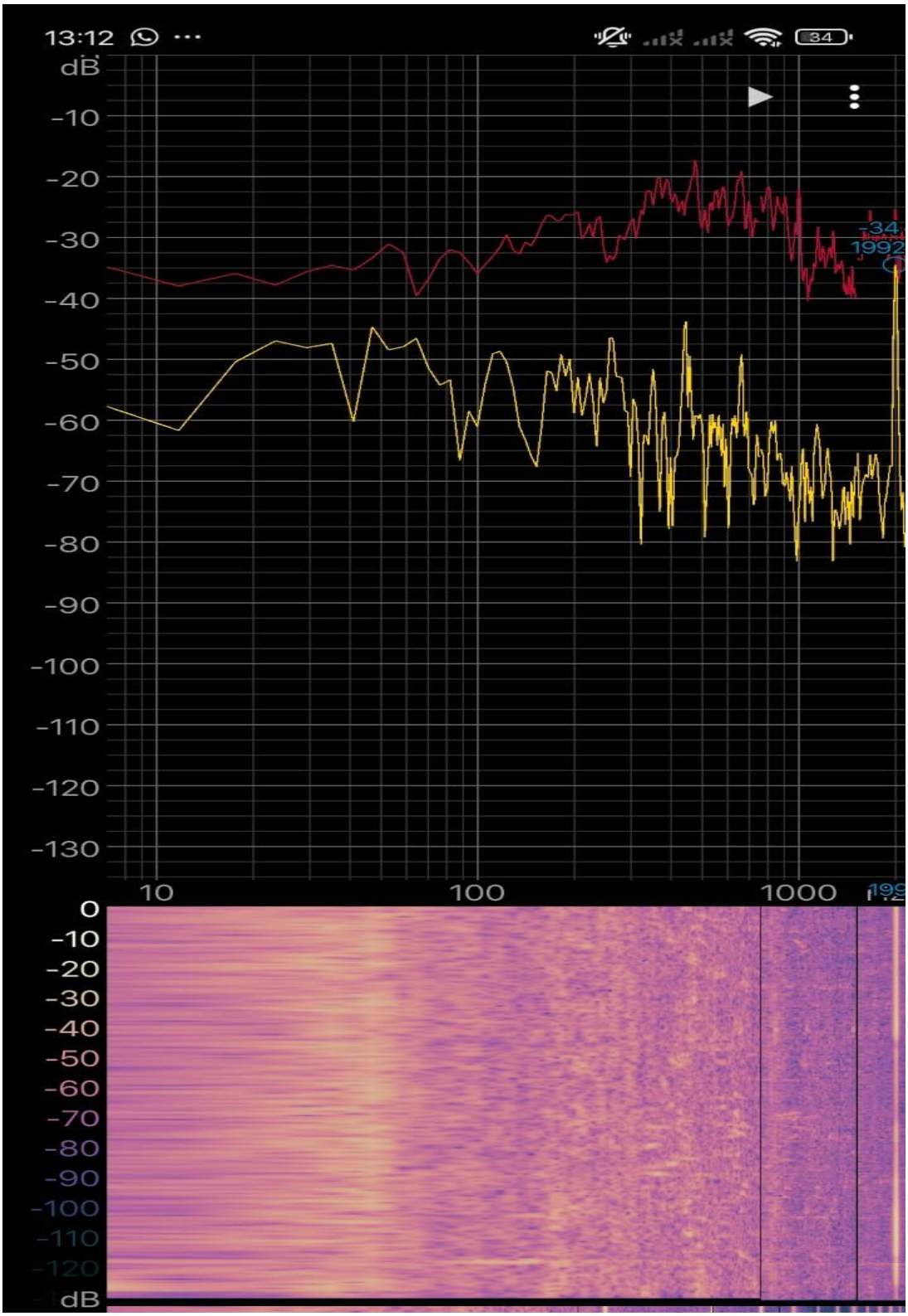
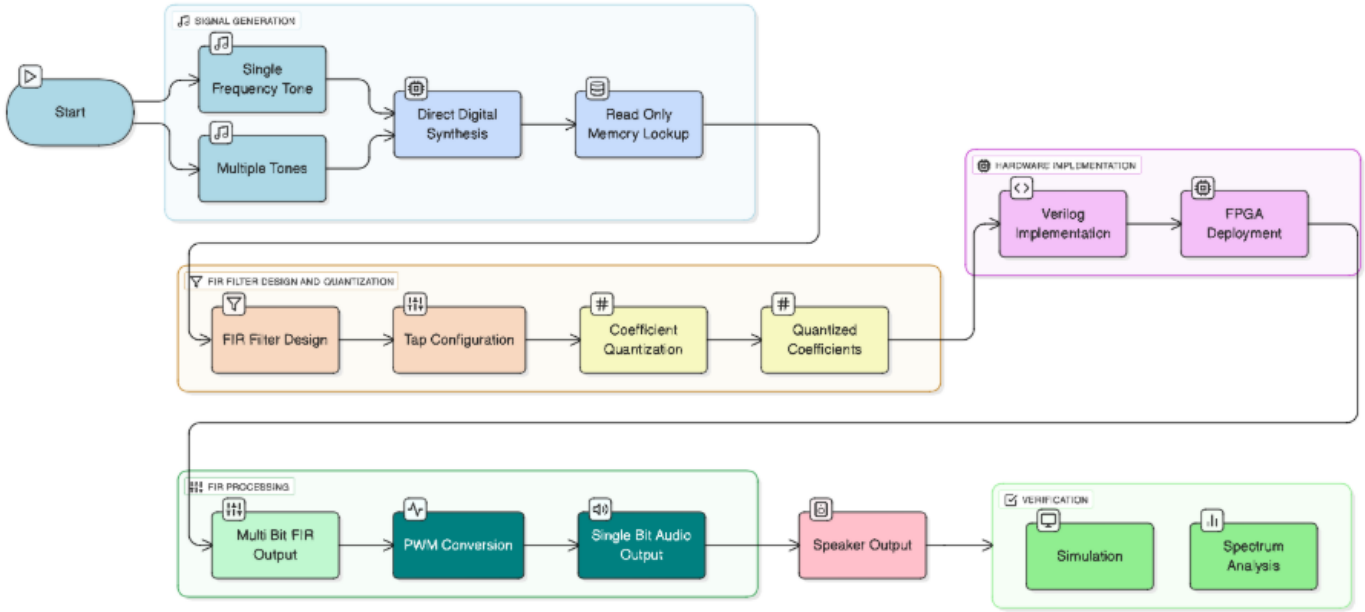
1 kHz Square Pulse Generator:

```
`timescale 1ns / 1ps

module square_wave_gen (
    input wire clk,          // 100kHz clock
    input wire reset,
    output reg signed [7:0] sq_out   // 8-bit signed output (-128 to 127)
);

    // For EXACT 1kHz square at 100kHz clock: 100 samples per cycle
    // 50 samples HIGH, 50 samples LOW
    reg [6:0] counter;  // 0 to 99 (7-bit for 100 values)

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            counter <= 7'd0;
            sq_out <= -8'd128;  // Start LOW (negative)
        end else begin
            // Count from 0 to 99 (100 values)
            if (counter == 7'd99)
                counter <= 7'd0;
            else
                counter <= counter + 7'd1;

            // Generate 1kHz square wave
            // First 50 cycles: HIGH (+127)
            // Next 50 cycles: LOW (-128)
            if (counter < 7'd50)
                sq_out <= 8'd127;    // Positive half
            else
                sq_out <= -8'd128;   // Negative half
        end
    end

endmodule
```

Unfiltered:


13:11
dB
-10
-20
-30
-40
-50
-55
-60
-70
-80
-90
-100
-110
-120
-130
-25 dB
996 Hz
-55 dB
352 Hz
10
100
352
1996
Hz
0
-10
-20
-30
-40
-50
-60
-70
-80
-90
-100
-110
-120
dB

Filtered:


13:12
dB
-10
-20
-30
-40
-50
-60
-70
-80
-90
-100
-110
-120
-130
10
100
1000
0
-10
-20
-30
-40
-50
-60
-70
-80
-90
-100
-110
-120
dB

Complete System Block Diagram: